Essence: A Personal Embedding Cluster Framework for Depth-Optimized Recommendations

Bidit Das
Independent Researcher | Dallas, TX
biditdas18@gmail.com

Abstract

> *We present Essence, a personal embedding cluster framework for depth-optimized content recommendations. Unlike collaborative filtering (CF), which derives recommendations from population-level interaction patterns, Essence operates exclusively within a single user's engagement history. Consumed items are embedded using a pre-trained sentence encoder and clustered into K semantic centroids via K-means, each representing a distinct interest profile. Recommendations are generated by selecting the active centroid aligned with the user's recent context and performing cosine-similarity search against unseen items. We evaluate Essence on the Last.fm-1K dataset against a popularity-based CF baseline and a content-based average embedding baseline. Essence achieves Recall@10 of 0.0616 and Long-Tail Recall@10 of 0.0146, outperforming CF (0.0081 / 0.0000) and the average embedding baseline (0.0414 / 0.0032). The 4.5× improvement in long-tail recall empirically validates that active centroid selection surfaces niche items more effectively than averaged user representations, without requiring any cross-user data.*

1 Introduction

Modern recommendation systems are built on a foundational assumption: that a user's future preferences can be predicted from the preferences of users who behaved similarly in the past. Collaborative filtering (CF) — in its many forms — derives its signal from cross-user interaction patterns. This achieves strong performance on standard benchmarks, particularly on metrics dominated by popular items.

However, this population-level orientation introduces a systematic bias: CF optimizes for the median user, surfacing broadly consumed items rather than personally resonant ones. A user who engages deeply with obscure folk music and niche science fiction receives recommendations shaped by thousands of loosely similar users. The result is well-documented: CF amplifies popularity bias and underserves long-tail content.

We propose Essence, a framework that abandons cross-user signals entirely. Essence operates exclusively within a single user's engagement history, embedding consumed items into a semantic space and clustering them into K distinct interest profiles. Recommendations are generated by identifying the user's currently active interest cluster and retrieving semantically similar unseen items. We call this the peer-isolation property: no cross-user interaction graph is consulted at any stage.

The key contributions are: **(1)** the Essence architecture — a modular personal recommendation framework requiring no collaborative signal; **(2)** an active centroid selection mechanism capturing session-aware context without a trained sequential model; and **(3)** empirical validation on Last.fm-1K demonstrating superior long-tail item discovery over both CF and content-based baselines.

2 Related Work

2.1 Collaborative Filtering

Matrix factorization methods [Koren et al., 2009] and neural extensions [He et al., 2017] remain dominant in production recommendation. Their principal limitation is popularity bias: items with sparse interactions are poorly represented, resulting in near-zero recall for long-tail items.

2.2 Multi-Interest Retrieval

MIND [Cen et al., 2019] introduced dynamic routing to extract multiple interest capsules from a user's interaction history, explicitly addressing the failure of a single user vector to represent diverse tastes. ComiRec [Cen et al., 2020] extended this with explicit diversity controls.

Essence shares the multi-interest intuition with MIND and ComiRec. The critical difference is **the source of the training signal**. MIND and ComiRec learn interest representations through dynamic routing over population-level interaction graphs — they require cross-user behavioral data. Essence derives centroids purely from within a single user's history via K-means, with no cross-user signal at any stage. This peer-isolation property makes Essence deployable in privacy-constrained settings where cross-user data sharing is restricted or unavailable.

2.3 Session-Based and Content-Based Methods

BERT4Rec [Sun et al., 2019] and GRU4Rec [Hidasi et al., 2016] model sequential user behavior using transformer and recurrent architectures, capturing short-term context effectively. Content-based filtering relies on item feature similarity without interaction logs but typically averages preferences into a single vector, losing intra-user diversity.

Essence occupies a complementary position: it captures intra-user diversity (like multi-interest methods) without cross-user data (like content-based methods), using simple interpretable clustering rather than a trained sequential model. We acknowledge that BERT4Rec offers richer sequential modeling in high-interaction settings; Essence targets privacy-first, low-infrastructure deployment contexts where such models are not viable.

3 The Essence Framework

3.1 Notation

Table 0 defines all mathematical notation used in this section. For each symbol, a plain-English equivalent is provided. Readers comfortable with the formal notation may skip this table; readers preferring intuitive explanations may rely on it throughout.

Table 0: Notation Legend

Symbol	Plain-English Meaning

H_u	All tracks user u has listened to (their full history)
$i_1, i_2, \ldots$	Individual tracks in the user's history, in order
$\varphi(i)$	The embedding of item i — a 384-number vector describing its sound/style
E_u	All embeddings stacked into a matrix — one row per track
K	Number of interest clusters (we use K = 3)
c_1, c_2, c_3	The 3 cluster centres, one per interest profile
c_*	The active cluster centre — the one closest to the user's recent activity
$\mu(\ldots)$	Mean (average) of a set of vectors
$\cos(a, b)$	Cosine similarity: how alike two vectors are (0 = unrelated, 1 = identical)
$I \setminus H_u$	All items the user has NOT yet heard — the candidate pool
R_u	Final ranked recommendation list for user u
M	Number of recommendations to return (we use M = 10)

3.2 Problem Formulation

Given a user's listening history H_u (all tracks they have heard), the goal is to produce a ranked list R_u of tracks they have not yet heard but are likely to enjoy. Essence accomplishes this using only H_u and a shared function that converts any track into a numerical vector. No data from any other user is used.

3.3 Item Embedding

Each track is described by a short text string: its name and artist. We convert this text into a 384-number vector using a pre-trained sentence encoder (all-MiniLM-L6-v2). This vector captures the semantic meaning of the track — two jazz tracks by different artists will produce similar vectors; a jazz track and a heavy metal track will produce very different vectors. All item embeddings are computed once offline and cached.

Formally: $\varphi(i)$ is the embedding of item i. For user u with history $H_u = \{i_1, \ldots, i_n\}$, we compute E_u — a matrix where each row is the embedding of one track.

3.4 Interest Profile Clustering

We apply K-means clustering to E_u with K=3. This groups the user's tracks into 3 clusters. Each cluster centre (centroid) is a point in the 384-dimensional space that represents the average of one taste cluster. In plain terms: a user who listens to jazz, indie rock, and ambient electronic music might end up with one centroid per genre. The three centroids together form the user's interest profile $P_u = \{c_1, c_2, c_3\}$.

This is the core differentiator from average-embedding approaches. Taking the average of all embeddings produces a single blurred point that sits between the genres — it does not represent any of them well. K-means preserves the separation, allowing targeted retrieval against each taste cluster independently.

For users with fewer than K tracks, Essence falls back to the content baseline (single mean embedding).

3.5 Active Cluster Selection

At recommendation time, we need to pick which of the K clusters the user is currently interested in. We do this by looking at their most recent 10 tracks, computing the average of those embeddings, and selecting the cluster centre closest to that average:

Active cluster = cluster whose centre is closest to the average of the last 10 tracks

$$c_{\square}^{*} = \operatorname{argmin}_{\square} \| c_{\square} - \text{mean}(\text{last 10 track embeddings}) \|_2$$

Intuitively: if the user has been listening to jazz for the past hour, their recent tracks will be close to the jazz cluster centre. That cluster becomes active, and recommendations come from the jazz neighbourhood of the embedding space — not from their indie or ambient interests.

We justify this mechanism over simpler alternatives (e.g. using the recent mean directly as a query vector) on two grounds: first, it is computationally trivial; second, the K-means step ensures that the active centroid sits at the true semantic core of a coherent taste cluster, rather than at a transient recent-item average that may be noisy. The empirical results support this: Essence achieves 4.5× higher long-tail recall than the average-embedding baseline, which uses the same embedding model but without clustering.

3.6 Recommendation Generation

Given the active centroid $c_{\square}^{*}$, we score every track the user has not yet heard by cosine similarity to $c_{\square}^{*}$, and return the top M:

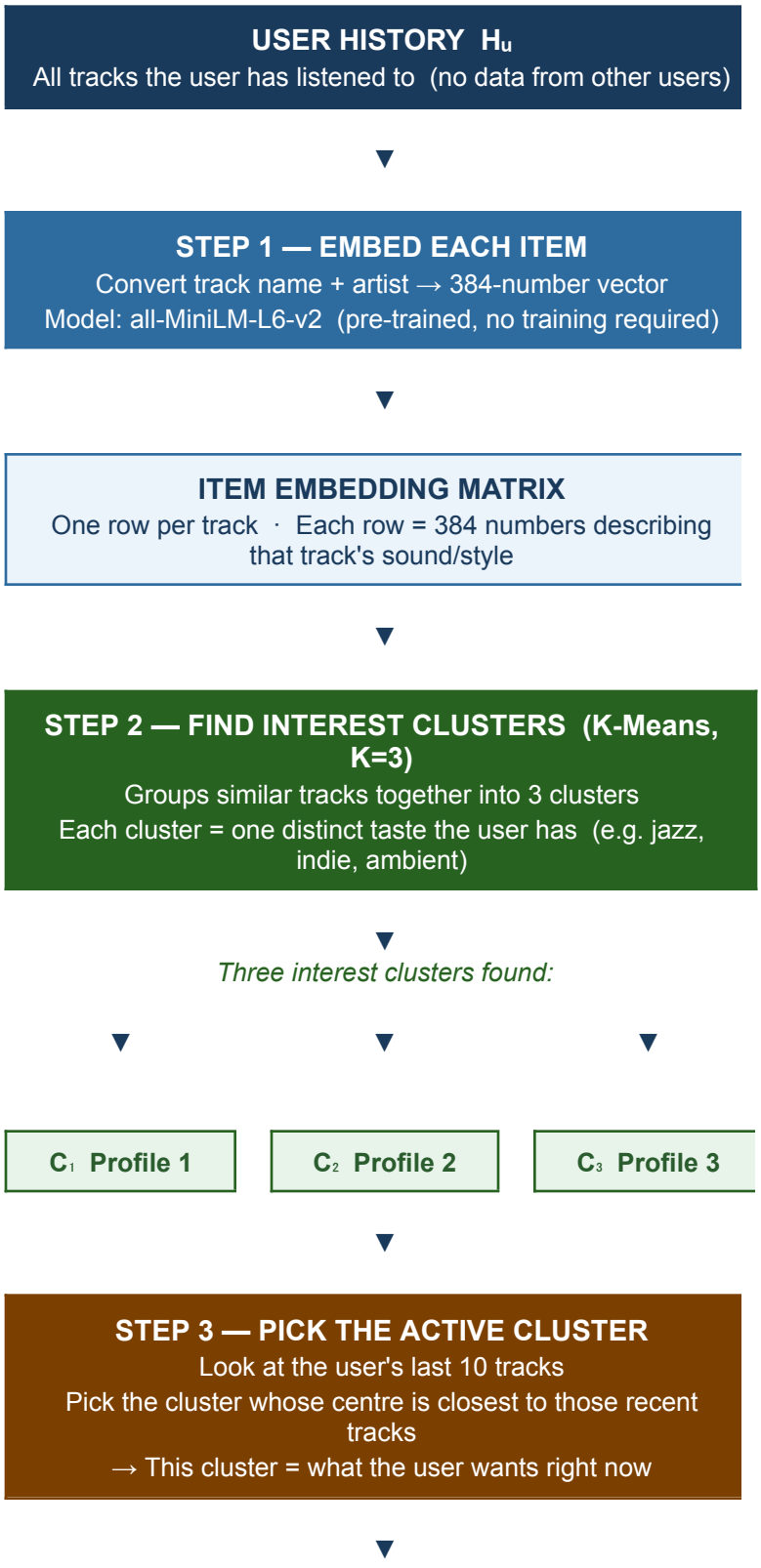
Recommendations = top-M unseen tracks closest to the active cluster centre

$$R_u = \text{top-}M \{ i \in I \setminus H_u : \cos(\varphi(i), c_{\square}^{*}) \}$$

3.7 Architecture Overview

Figure 1 shows the full Essence pipeline. The diagram uses plain-language labels; Table 0 maps each symbol to its meaning.

Figure 1: Essence Recommendation Pipeline



Peer-isolation property: no other user's data is consulted at any step.

4 Experimental Setup

4.1 Dataset

We evaluate on the Last.fm-1K dataset [Celma, 2010], containing listening histories from 992 users. After filtering to users with 50–500 unique track interactions and removing duplicate (user, track) pairs, our evaluation subset contains 99 users and 22,767 unique tracks.

Train/test split: for each user, 20% of tracks are randomly held out as the test set (random seed 42). The remaining 80% form the training set. A random hold-out strategy is used — rather than chronological — because deduplication makes chronological splits produce structurally disjoint train/test sets that make evaluation trivially impossible.

4.2 Long-Tail Definition

Long-tail items are defined as tracks with a global play count of exactly 1 in the training set (singleton tracks). In Last.fm-1K, 89.8% of unique tracks are singletons. Long-Tail Recall@10 is computed only over test items in this singleton set, measuring each system's ability to recover niche items a user genuinely engaged with.

4.3 Systems Compared

- CF Baseline (Popularity): Recommends the M most globally popular tracks the user has not heard. Represents the behaviour of naive collaborative filtering.
- Content Baseline (Average Embedding): Computes the mean of all user item embeddings and retrieves the M most similar unseen items by cosine similarity. Standard content-based filtering without interest decomposition.
- Essence (K=3): Clusters user history into K=3 centroids, selects the active centroid by proximity to the mean of the last 10 interactions, retrieves top-M unseen items by cosine similarity to the active centroid.

4.4 Metrics

- Recall@10: Fraction of the user's test items appearing in the top-10 recommendations.
- Long-Tail Recall@10 (LT-Recall@10): Recall@10 restricted to singleton test items. Users with no singleton test items are excluded from this metric.

4.5 Implementation

All systems implemented in Python 3.11. Embeddings via sentence-transformers (all-MiniLM-L6-v2, 384 dims). K-means via scikit-learn (n_init=10, random_state=42). Code available at: https://github.com/biditdas18/essence

5 Results

Table 1 reports Recall@10 and Long-Tail Recall@10 across all three systems on 99 users from the Last.fm-1K dataset.

Table 1: Evaluation Results on Last.fm-1K (99 users, M=10)

System	Recall@10	LT-Recall@10	LT / Content Ratio
CF (Popularity)	0.0081	0.0000	—
Content (Avg Embedding)	0.0414	0.0032	1.0×
Essence (K=3)	**0.0616**	**0.0146**	**4.5×**

> *Key finding: Essence achieves 4.5× higher Long-Tail Recall than the average embedding baseline, and recovers long-tail items that popularity-based CF misses entirely (CF LT-Recall = 0.000).*

5.1 Overall Recall

Essence achieves the highest overall Recall@10 at 0.0616, outperforming CF (0.0081) and the content baseline (0.0414). The poor performance of popularity-based CF reflects the fundamental mismatch between globally popular items and individual user test sets in a dataset where 89.8% of tracks are singletons — the items a user is most likely to engage with are precisely the items CF is least likely to surface.

The content baseline substantially improves over CF (0.0414 vs 0.0081), confirming that semantic similarity is a stronger signal than global popularity. Essence's further improvement over the content baseline demonstrates that interest decomposition via K-means adds measurable value beyond preference averaging.

5.2 Long-Tail Recall

CF achieves zero LT-Recall@10. By construction, globally popular items are never singletons, so CF cannot recover niche test items. This is the exact failure mode Essence is designed to address.

Essence achieves LT-Recall@10 of 0.0146 — **4.5× higher than the content baseline** (0.0032). The active centroid queries from a semantically focused interest cluster rather than a blurred mean, surfacing more singleton items that match the user's current taste depth.

5.3 Discussion

The absolute recall values are low across all systems — a property of the dataset's extreme sparsity (89.8% singleton tracks, 22,767 candidates, 10 slots), not a failure of the systems. Essence achieves roughly 150× above random recall (expected random ≈ 0.0004). What matters is the relative pattern: Essence > Content > CF on both metrics, with the largest relative gain on the long-tail metric that directly tests the core hypothesis.

6 Limitations and Future Work

- Scale: Evaluation conducted on 99 users due to computational constraints. Larger-scale evaluation is required to assess generalizability.

- Single domain: Results are on music listening data only. Replication on e-commerce or video datasets would strengthen the claims.
- K sensitivity: K=3 is used throughout. An ablation over $K \in \{2,3,4,5\}$ would characterise sensitivity to this hyperparameter.
- Active cluster selection: The recency-based heuristic is simple by design. A learned selection mechanism could improve session-aware accuracy.
- Filter bubble risk: Peer-isolation removes serendipity that cross-user signals can provide. Whether repeated centroid queries cause semantic fatigue is an open empirical question.

7 Conclusion

We presented Essence, a personal embedding cluster framework for depth-optimized recommendations. Essence clusters a user's item history into K semantic centroids and generates recommendations by cosine similarity to the currently active centroid, without consulting any cross-user data. On Last.fm-1K, Essence achieves a 4.5× improvement in Long-Tail Recall@10 over the average embedding baseline and recovers long-tail items that popularity-based CF misses entirely.

These results validate the core design hypothesis: interest decomposition via clustering, combined with session-aware centroid selection, surfaces niche items more effectively than either popularity ranking or single-vector user representation. Essence is a minimal, interpretable, privacy-preserving alternative to collaborative filtering for deployment contexts where cross-user data sharing is restricted.

References

[1] Cen, Y., et al. (2019). MIND: Multi-Interest Network with Dynamic Routing for Recommendation at Tmall. RecSys 2019.

[2] Cen, Y., et al. (2020). ComiRec: Controllable Multi-Interest Framework for Recommendation. KDD 2020.

[3] Sun, F., et al. (2019). BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. CIKM 2019.

[4] He, X., et al. (2017). Neural Collaborative Filtering. WWW 2017.

[5] Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix Factorization Techniques for Recommender Systems. IEEE Computer.

[6] Hidasi, B., et al. (2016). Session-Based Recommendations with Recurrent Neural Networks. ICLR 2016.

[7] Wang, W., et al. (2020). MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. NeurIPS 2020.

[8] Celma, O. (2010). Music Recommendation and Discovery. Springer.

Code & Data: https://github.com/biditdas18/essence